
From a Small Language Model to AI Agent

Mentee: Ziyu Liao Mentor: Yuhan Qian

A Semester Review Report
DRP · Spring 2026



“By the end of this course, you will have trained your own small language model from scratch — understanding every component along the way.”

1. Course Overview

This report summarizes the key concepts covered in this quarter, a ten week course. We built and trained *MiniMind* — a small but fully functional language model — completely from scratch using PyTorch. We follow a natural progression: from raw text to tokens, from tokens to vectors, from vectors to a full Transformer, and finally from a raw model to an instruction-following assistant.

Week	Topic
1	Introduction & PyTorch warm-up
2	Tokenization (BPE)
3	Embeddings & Positional Encoding (RoPE)
4	Self-Attention Mechanism
5	The Full Transformer Block
6	Pretraining
7	Supervised Fine-Tuning (SFT)
8	Evaluation & Inference
9	LoRA Fine-Tuning
10	DPO & Retrospective

The following sections walk through the most important concepts, grouped by theme.

2. From Text to Numbers: Tokenization

Before a neural network can process language, text must be converted into **numbers**. The process is called *tokenization*: splitting a string into smaller units called *tokens* and mapping each token to an integer ID.

2.1 Byte-Pair Encoding (BPE)

MiniMind uses **Byte-Pair Encoding (BPE)**, the same family of algorithm used by GPT-2 and many modern LLMs. BPE finds a sweet spot between two extremes:

- *Character-level*: sequences are very long; the model must learn to combine characters into words.
- *Word-level*: the vocabulary explodes and rare words become “unknown” (<unk>).

BPE starts with individual bytes and **iteratively merges** the most frequently co-occurring pair of symbols, building up a vocabulary of common sub-words. Because every character can be represented as a sequence of bytes, byte-level BPE guarantees **zero unknown tokens** — any input, including code, numbers, or emoji, can always be tokenized.

Key design choice: MiniMind uses a vocabulary of **6,400 tokens**. A larger vocabulary produces shorter sequences but requires a bigger embedding table; 6,400 keeps the total model small (≈ 26 M parameters).

Special tokens such as <|im_start|> and <|im_end|> are manually added to structure multi-turn conversations during fine-tuning.

3. From IDs to Vectors: Embeddings & Positional Encoding

A token ID is just an integer — it carries no semantic information. **Embeddings** solve this by mapping each integer to a dense, learnable vector that gradually encodes meaning through training.

3.1 Token Embeddings

The embedding table is a matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$, where V is the vocabulary size and d is the model dimension. Looking up token i simply selects row i of $\mathbf{E}$:

$$\mathbf{x}_i = \mathbf{E}[i] \in \mathbb{R}^d$$

After training, similar tokens (e.g. “king” and “queen”) end up close to each other in this d -dimensional space.

3.2 Why Positional Encoding?

A plain embedding table treats “dog bites man” and “man bites dog” identically — both are just a bag of token IDs. **Positional encoding** fixes this by injecting position information so the model knows *where* each token appears in the sequence.

3.3 Sinusoidal Positional Encoding

The original Transformer (Vaswani et al., 2017) uses a fixed, non-learnable sinusoidal encoding:

$$\text{PE}_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

Each position receives a unique vector; different dimensions oscillate at different frequencies. The encoding is *added* to the token embedding: $\mathbf{x}' = \mathbf{x} + \text{PE}_{\text{pos}}$.

3.4 RoPE: Rotary Position Embedding

MiniMind uses the more modern **RoPE** (Su et al., 2021), which encodes position by *rotating* the query and key vectors rather than adding a fixed offset. Pairs of dimensions are treated as 2D vectors and rotated by an angle θ that depends on the position:

$$(R(\theta_m) \mathbf{q})^\top (R(\theta_n) \mathbf{k}) = \mathbf{q}^\top R(\theta_m - \theta_n)^\top \mathbf{k}$$

The key insight: the attention score depends on the *relative* position ($m - n$), not on the absolute positions m and n individually. RoPE requires **no extra parameters** and naturally handles sequences longer than those seen during training.

4. The Heart of the Transformer: Attention

Self-attention is the mechanism that allows every token in a sequence to “look at” every other token and decide how much information to gather from each one.

4.1 Scaled Dot-Product Attention

Given an input sequence, three projection matrices W_Q , W_K , W_V produce **Queries**, **Keys**, and **Values**:

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V$$

Think of it like a search engine: the query is *what you’re looking for*, the keys are *labels on each memory slot*, and the values are the *actual content* stored there.

The output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The four steps are: (1) **score** — compute $QK^\top$ to see which keys match the query; (2) **scale** — divide by $\sqrt{d_k}$ to keep gradients healthy; (3) **mask** — set future positions to $-\infty$ so the model cannot “cheat” by looking ahead; (4) **aggregate** — multiply the softmax weights by V to form the output.

4.2 Multi-Head Attention (MHA)

Instead of running attention once, we run it h times in parallel with different projections, then concatenate:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O,$$

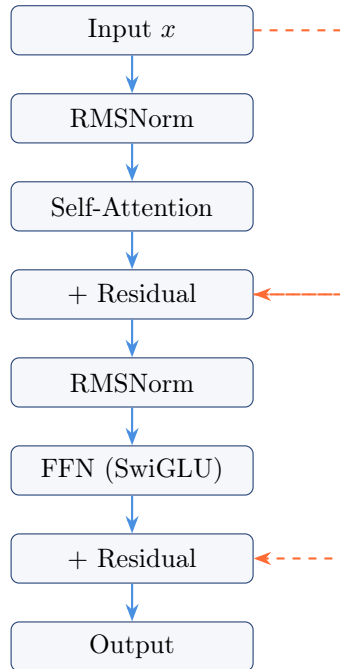
where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. Different heads can simultaneously attend to different aspects of the sequence — one head may track syntactic dependencies while another handles coreference.

4.3 Grouped Query Attention (GQA)

MiniMind uses **Grouped Query Attention**, a memory-efficient variant. Instead of one K/V pair per head (MHA), g groups of query heads share a single K/V pair. With $h = 8$ query heads and $g = 2$ groups, the KV-cache is $4\times$ smaller than full MHA — a significant saving during inference.

4.4 The Full Transformer Block

Each Transformer block stacks attention with a **Feed-Forward Network (FFN)** and uses **RMSNorm** and **residual connections**:



The FFN expands the dimension by $4\times$, applies a non-linearity, then projects back:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1) W_2 + b_2.$$

MiniMind uses the **SwiGLU** variant for smoother gradients. Residual connections prevent vanishing gradients, while **RMSNorm** stabilises training — the full model simply stacks N of these blocks.

5. Training the Model

5.1 Pretraining: Next-Token Prediction

Pretraining is **self-supervised**: the model learns from raw text by predicting each next token given all previous ones. No human labels are needed — the labels come from the text itself.

The loss function is the **cross-entropy** between the model’s predicted distribution and the true next token:

$$\mathcal{L} = - \sum_{t=1}^T \log P_{\theta}(x_{t+1} \mid x_1, \dots, x_t)$$

Equivalently, for a single position with true label y and predicted distribution q :

$$H(p, q) = - \sum_x p(x) \log q(x)$$

A low loss means the model assigns high probability to the correct token; a high loss means the model is “surprised” by what comes next.

Training uses the standard loop: *forward pass* $\rightarrow$ *compute loss* $\rightarrow$ *backward pass* $\rightarrow$ *optimizer step*, repeated over billions of tokens with a cosine learning rate schedule and gradient accumulation.

5.2 Evaluation: Perplexity

Perplexity (PPL) summarises how surprised the model is on held-out text:

$$\text{PPL} = \exp(H) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log p(x_i \mid x_{<i})\right)$$

A perfect model would have $\text{PPL} = 1$; a model that assigns uniform probability over a 50k vocabulary would have $\text{PPL} \approx 50,000$. Lower is better.

5.3 Supervised Fine-Tuning (SFT)

After pretraining, the model knows how to *complete text* but does not know how to *follow instructions*. SFT bridges that gap by continuing training on curated **instruction–response pairs**:

Aspect	Pretraining	SFT
Data format	Raw text	Instruction–response JSONL
Loss	All tokens	Response tokens only
Learning rate	$\sim 10^{-4}$	$\sim 10^{-5}$ (10 $\times$ smaller)
Epochs	1 (single pass)	2–5 (multiple passes)
Dataset size	Billions of tokens	Thousands of examples

A crucial technique is **loss masking**: during SFT the cross-entropy loss is only computed on the *assistant’s response tokens*, not on the user’s prompt. This focuses the gradient on teaching *how to reply*, not on re-learning the prompt distribution.

6. Efficient Fine-Tuning with LoRA

Full fine-tuning updates every parameter of the model — expensive in both memory and compute. **LoRA** (Low-Rank Adaptation, Hu et al., 2021) offers a dramatically cheaper alternative.

6.1 The Key Insight: Low-Rank Structure

Research by Aghajanyan et al. (2020) showed that pretrained models have a low *intrinsic dimensionality*: the useful changes during fine-tuning live in a small subspace. Even though a weight matrix $W \in \mathbb{R}^{d \times d}$ has d^2 entries, the “direction” of adaptation can be captured with far fewer parameters.

6.2 LoRA Parameterisation

Instead of updating W directly, LoRA freezes it and adds a *low-rank bypass*:

$$W_{\text{new}} = W_{\text{frozen}} + \underbrace{A \cdot B}_{\Delta W}, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times d}, \quad r \ll d$$

Only A and B are trained. The matrix B is initialised to zero so the adapter has no effect at the start of training. A scalar α/r controls the magnitude of the update.

6.3 Parameter Savings in Practice

For a single linear layer with $d = 512$:

Method	Trainable params	Ratio
Full fine-tuning	$512 \times 512 = 262,144$	1×
LoRA ($r = 4$)	$2 \times 512 \times 4 = 4,096$	64× smaller
LoRA ($r = 1$)	$2 \times 512 \times 1 = 1,024$	256× smaller

For a 7B-parameter model with $r = 8$, only $\approx 0.1\%$ of parameters are trainable. Optimizer states (Adam momentum and variance) only need to be stored for those parameters, dramatically reducing GPU memory.

Which layers get LoRA?

In the attention block, $Q = xW_Q$, $K = xW_K$, $V = xW_V$, $O = \text{Attn}(Q, K, V) \cdot W_O$. Adapting W_Q and W_V captures most of the task-specific signal. Start with these two; add more layers only if performance demands it.

7. Summary & Key Takeaways

Over ten weeks we traced the full lifecycle of a language model:

1. **Tokenization (BPE)** converts raw text into integer IDs with a compact, lossless vocabulary.
2. **Embeddings** map token IDs to learnable dense vectors; RoPE injects relative position information by rotating Q and K vectors before the attention dot product.
3. **Scaled dot-product attention** lets every token attend to every other; multi-head and grouped-query variants balance quality against memory.
4. **The Transformer block** stacks attention, FFN, RMSNorm, and residual connections; stacking N blocks forms the full model.
5. **Pretraining** on next-token prediction with cross-entropy loss teaches the model language statistics over billions of tokens.
6. **SFT** fine-tunes the pretrained model on instruction–response pairs with loss masking, turning a text-completer into an instruction-follower.
7. **LoRA** makes fine-tuning cheap by freezing the base model and training only tiny low-rank adapter matrices.

Together, these components explain how modern large language models such as GPT, LLaMA, and MiniMind work under the hood. The course demonstrated that none of these ideas requires black-box magic — with PyTorch and the concepts above, you can build and train your own language model from scratch.

The Big Picture

$Text \xrightarrow{\text{BPE}} Token\ IDs \xrightarrow{\text{Embedding} + \text{RoPE}} Vectors \xrightarrow{\text{Transformer Blocks}} Logits \xrightarrow{\text{SFT} + \text{LoRA}} Assistant$